

Git & GitHub — Complete Study Notes

Beginner to Advanced Version Control

A practical reference covering Git fundamentals, repositories, commits, branches, .gitignore, remote repositories, collaboration, merge conflicts, stash, merge vs rebase, GitHub workflows, security, and useful commands.

1. What is Version Control?

Version control records changes to files over time so you can review history, restore earlier versions, collaborate, and work safely on different features.

2. Git vs GitHub

Git is a distributed version control system that runs locally. **GitHub** is a cloud platform for hosting Git repositories and collaborating around them. Git can be used without GitHub.

3. Centralized vs Distributed Version Control

Centralized systems depend on a central server for the main repository. Git is distributed: each clone contains repository history, allowing many operations locally and making branching lightweight.

4. Git Installation and Identity

After installing Git, configure the identity used for commits. Use a name and email appropriate for your development setup.

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --list
```

5. Repository Basics

A Git repository stores project history and metadata. Initialize a new repository with `git init`, or copy an existing repository with `git clone`.

```
git init
git clone <repository-url>
git status
```

6. The Git Working Areas

Think of Git as three important areas: **working directory** (your files), **staging area** (changes selected for the next commit), and **repository history** (committed snapshots).

```
git status
git add file.py
git commit -m "Add employee validation"
git log
```

7. Basic Daily Workflow

A common workflow is: edit → inspect → stage → commit → push. Keep commits focused and use meaningful messages.

```
git status
git diff
git add .
git status
git commit -m "Add input validation"
git push
```

8. Tracking and Untracking Files

Git tracks files after they are staged and committed. `git add` stages changes. To stop tracking a file while keeping it locally, use `git rm --cached`.

```
git add file.py
```

```
git add .
git rm --cached config.txt
git status
```

9. .gitignore

A .gitignore file tells Git which files or patterns should normally not be tracked. Common examples include virtual environments, caches, IDE files, build output, and secrets.

```
.venv/
__pycache__/
*.pyc
.env
.vscode/
```

10. Never Commit Secrets

Never push passwords, API keys, database credentials, private tokens, or sensitive environment files to a public repository. Use environment variables or secret-management systems. If a secret is exposed, rotate/revoke it; deleting the file in a later commit does not necessarily remove it from Git history.

11. Commits

A commit is a snapshot of staged changes with metadata and a message. Good commits are small, focused, and descriptive.

```
git add .
git commit -m "Add employee search feature"
git log --oneline
git show <commit>
```

12. Viewing Changes

Use status to see the state of the working tree, diff to inspect unstaged changes, and diff --staged to inspect staged changes.

```
git status
git diff
git diff --staged
git log --oneline --decorate --graph
```

13. Branches

A branch is an independent line of development. Branches let you work on features or fixes without disturbing the main branch.

```
git branch
git switch -c feature/login
git switch main
git merge feature/login
git branch -d feature/login
```

14. Branching Workflow

A simple feature workflow: update main → create feature branch → develop → commit → push branch → open a GitHub pull request → review → merge.

```
git switch main
git pull
git switch -c feature/new-report
# edit files
git add .
git commit -m "Add new report"
git push -u origin feature/new-report
```

15. Remote Repositories

A remote is a repository hosted elsewhere, commonly on GitHub. origin is the conventional name for the main remote.

```
git remote -v
```

```
git remote add origin <repository-url>
git push -u origin main
git pull origin main
```

16. GitHub Pull Requests

A pull request proposes changes from one branch into another. It supports review, discussion, automated checks, and controlled merging. In team projects, avoid pushing directly to protected main branches when the project uses PR-based workflow.

17. Fetch vs Pull

git fetch downloads remote information without changing your current branch. git pull generally fetches and then integrates the remote changes into the current branch.

```
git fetch origin
git pull origin main
```

18. Merge Conflicts

A conflict occurs when Git cannot automatically combine competing changes. Resolve the conflicting file manually, then stage the resolved file and commit the merge.

```
git status
# edit conflicted files
git add <resolved-file>
git commit
```

19. Stash

Stashing temporarily stores uncommitted changes so you can switch context without committing unfinished work. It is useful for short-lived interruptions.

```
git stash
git stash list
git stash pop
git stash apply
```

20. Merge vs Rebase

Merge combines histories and may create a merge commit. **Rebase** moves/replays commits onto another base to create a more linear history. Avoid rebasing shared/public history unless your team explicitly agrees.

```
git switch feature
git merge main

git switch feature
git rebase main
```

21. Undoing Changes Safely

Different commands undo different things. restore can discard working-tree changes; reset moves branch pointers and can change staging/history; revert creates a new commit that reverses an earlier commit. Be especially careful with history-rewriting commands.

```
git restore file.py
git restore --staged file.py
git revert <commit>
git reset --soft HEAD~1
```

22. GitHub Desktop

GitHub Desktop provides a graphical interface for common Git operations such as viewing changes, committing, switching branches, pulling, pushing, and creating pull requests. Understanding command-line Git first makes GUI tools easier to understand.

23. Collaboration

For team work, communicate branch purpose, make focused commits, pull/fetch regularly, review changes through pull requests, resolve conflicts carefully, and avoid committing generated or secret files.

24. Useful Commands Cheat Sheet

Keep this section as a quick reference.

```
git status
git init
git clone <url>
git add <file>
git add .
git commit -m "message"
git log --oneline
git diff
git branch
git switch -c <branch>
git switch <branch>
git merge <branch>
git fetch
git pull
git push
git stash
git stash pop
git remote -v
git revert <commit>
```

25. Recommended Beginner Practice

Create a small practice repository. Make a file, commit it, modify it, inspect the diff, create a feature branch, commit the feature, push it to GitHub, open a pull request, and practice resolving a small conflict. Then practice stash and understand merge vs rebase.

26. Interview Quick Revision

Git: distributed version control system. **GitHub:** hosting/collaboration platform for Git repositories. **Commit:** saved snapshot of staged changes. **Branch:** independent line of development. **Remote:** another repository location. **Staging:** selecting changes for the next commit. **Merge:** integrates histories. **Rebase:** replays commits onto another base. **Stash:** temporarily shelves uncommitted changes. **.gitignore:** specifies files Git should normally ignore.

27. Practical Git Workflow to Memorize

```
# Start or clone
git clone <url>
cd project

# Daily work
git status
git switch -c feature/my-feature
# edit files
git diff
git add .
git commit -m "Describe the change"

# Sync and publish
git pull --rebase
git push -u origin feature/my-feature

# Finish through GitHub
# Open Pull Request → review → checks → merge
```

28. Golden Rules

- Commit often, but keep commits logically focused.
- Write clear commit messages.
- Use branches for features and fixes.
- Review changes before committing and pushing.

- Never commit secrets or credentials.
- Use .gitignore for files that should not be tracked.
- Be cautious with force-push and history rewriting.
- Learn the command-line workflow even if you later use GitHub Desktop.

Key idea: Git is the version-control engine; GitHub is a collaboration and hosting platform built around Git. The most important skill is understanding the flow: working directory → staging → commit → branch → remote → collaboration.